AUTOMATED TRADING PLATFORM PROJECT UPDATE

Abdelaziz Hasan

CS 451

GENERAL MOTIVATIONS

In this project, we are aiming to build an automatic trading platform with distributed resources. The final implementation can be expected to include a refined UI for net gains and losses, latency and throughput metrics, both a distributed and a monolithic configuration of the programs, and optionally: subscription to some live feed of data.

CAVEATS

There has generally been no shortage of these. Firstly, this current implementation does not read live data. The heavy requirements of IP management with Fabric's hostile "support" made this difficult, and so it will be saved for a later version if at all implemented. In its place, market data is synthetically generated in the exact form that *websocket*, the module responsible for providing live market data, would send it in.

The second setback would be the implementation of a proper strategy. I am not a stockbroker, nor do I have any interest in that matter. The intention behind this project is to apply knowledge of distributed systems to use a collection of resources in a clever way. This does not include learning the nitty gritty on long/short moving averages. As such, the strategy used was grabbed from Stack Overflow as was tweaked very cautiously and faintly for testing purposes only.

The next obstacle is the monitor for the program. After a little thought on what may be a cohesive way to visual the programs out – the very same program that sees no real data and confirms no real trades – would be to simulate net positions and calculate the capital gains and losses for a few different stock options. Again, the data is generated so this information will be completely fictional, but the monitor updates continuously and displays information as it gets it. Thus far, logging efforts have been successful and can be used in the future for a more stylish UI with charts, but that is for another day. Currently, the only results presented are the latest ones.

CONFIGURATION

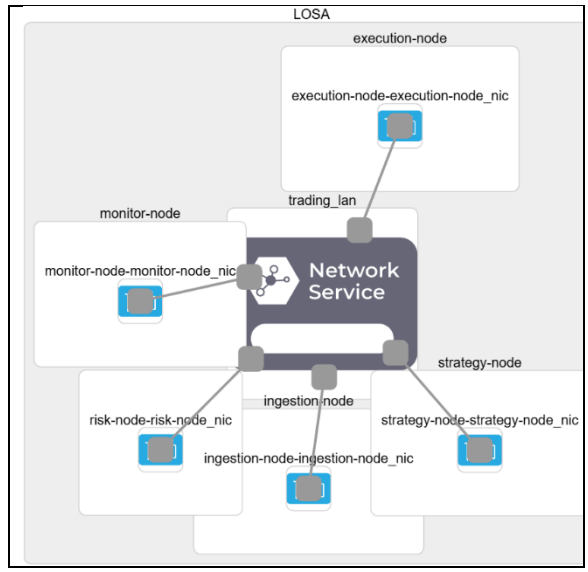

Figure 1: PNG depicting the slice configuration, generated by Fabric.

Models	Usage
class PriceTick(BaseModel): symbol: str price: float timestamp: str ingestion_ts: float = 0.0	Input data – produced in the ingestion node and consumed by the strategy node. Has four channels, one for each mock stock option. Ingestion_ts used for latency.
class Signal(BaseModel): signal_id: str symbol: str action: literal["buy", "sell"] confidence: float price_at_signal: float ingestion_ts: float	After analyzing MAs yielded from PriceTicks, a Signal will be produced if the moving average suggests a 'BUY' or 'SELL' event. Consumed by the risk node. Last two fields are carried throughout.
class Order(BaseModel): order_id: str symbol: str action: literal["buy", "sell"] quantity: int price_at_order: float ingestion_ts: float	Risk node subscribes to Signals, produces to Fills. Confirms a signal based on current position details, e.g. will not short when net position is less than negative five. Last two fields are curried forward.
class Fill(BaseModel): fill_id: str symbol: str action: literal["buy", "sell"] quantity_filled: int fill_price: float ingestion_ts: float	Order confirmation – generally useless in this simulation case but this would be the "receipts" of any confirmed trade by the execution node. Curried price is multiplied by N(1, 0.02) to mimic potential changes between ingestion and execution. Last field will be used for measuring latency.

Figure 2: Chart depicting the model class arrangements and abstract uses.

EXECUTION

Functional nodes include all but 'monitor-node', whose sole purpose is to display data consumed from the Fills bus.

Ingestion: At each market tick, of which there are two per second, a price is generated for all symbols from the distribution U(150, 200) and timestamps are grabbed using python's *time* module. This data is produced to PriceTicks and in the future, we will look into writing PriceTicks with live market data using *websocket*.

Strategy: Subscribed to PriceTicks, every price will be added to a window. With hyperparameters of five and twenty for the short and long window sizes, our node kicks off when twenty ticks are logged. Our small MA looks at the last five ticks in this queue and our long MA looks at the entire twenty. Any more ticks added will dequeue the oldest so that after our length first reaches twenty, we will never have more or less than twenty ticks. If $sMA > lMA * 1.005$, we will issue a "BUY" event to the Signal bus. If $sMA < lMA * 0.995$, we will issue a "SELL" event to the Signal bus. The price and timestamp from the PriceTick message is curried and the symbol is also logged.

Risk: We use simple logic to approve events here. Subscribed to the Signal bus, an Order will be approved only if a "BUY" event occurs when our net position is less than five, and a "SELL" event occurs when our net position is greater than negative five. The idea is that we don't wish to hoard stocks, but this gets complicated when the latency of the program is considered. The dashboard output seems to ignore this logic since messages are stockpiled and concurrently approved. It's a distributed race condition. These Orders are produced and curry forward the ingestion prices and timestamp.

Execution: This node would be responsible for closing trades with some API, but for lack of funds and courage, I left this node a little bleak and have it writing the Fill messages from the subscribed Order messages. All fields are filled accordingly, but the curried price field needs some changes. Ordinarily, this could be grabbed from the trading API as soon as a transaction was closed. We would expect slight differences between the execution price and the ingestion price, due to the latency. To better model this relationship, I multiply the curried ingestion price by N(1, 0.02) to introduce a little market chaos. Now, we can see the changes we expect to observe when trading.

DEPLOYMENT

The environment includes python files for four functional nodes, another two for the monitor and models, a .gitignore that blacklists the project_id file containing the Fabric project_id key, and the Jupyter notebook that deploys all the files.

Dependencies & Fabric Configuration: Here we import the modules needed in the notebook, as well as initialize the slice.

Node Configuration: This section allocates some hardware for five different nodes: ingestion, strategy, risk, execution, and the monitor node. Early hurdles proved it was beneficial to allocate at least four GB of memory per node. This section also installs the necessary dependencies on each node. The model uses *pydantic* so all nodes need it, as well as *confluent-kafka* and *redis*. My "chaos" with the execution price requires *numpy* on the execution node only. Beyond this, the ingestion node needs to extract files for *kafka* and deploy them before any scripts run, as well as install additional *redis-server* packages. All nodes get their respective python script uploaded here as well.

Deployment: We open with a "nuclear clean" procedure, it was used excessively when experimenting with different strategies in this project. Its role is to shut down/flush out any message bus or server data, as well as removing any scripts or logs from all nodes before reuploading them. We also adjust the IPs to use internal 10.0.0.* indexed IPs, with the ingestion node at 10.0.0.1, so that all nodes know where to produce and consume events to. Then we start *redis* and *kafka* up before running the main script on each node in their backgrounds.

Validation: Here is a little section for peeking at the message bus or the logs on each node, and it even includes the ssh command to log into the monitor. Inside the monitor, you need to run:

'python3 dashboard.py'

This will start the compact UI that displays the current state. It will not yield any output for around twenty seconds after the scripts were deployed but will continuously update after that. Running tallies start from zero but log information from messages passed even before the dashboard was booted up.

OUTLOOK

A few things need to be done before I can wrap this project up:

We lack any real benchmarking here. I have the infrastructure in place to start collecting latency, but currently do nothing with it. It's not even displayed in the dashboard. I need to think of some facet to display this information, as well as develop some support for measuring throughput.

We lack any purpose to benchmark as well. So far, we have only the distributed implementation of the program when we planned on having a monolithic implementation as well. It wouldn't be too hard to adapt what I have into a non-distributed setup, but this too is on the agenda. I plan to benchmark between the two implementations and note of any differences, however opaque they obviously are currently. It might be interesting to see if we can do seeded runs between the two and observe how the latency affects the net profit.

I want to further enhance the UI that's displayed in the monitor node. By reading the last fifty-or-so events in the monitor-node.log, I can make a queue and plot some charts for net gains and losses in each stock option. Paired with the benchmarking, this could present some exciting findings on the martingale nature of the stock market.

OPTIONAL: It would be possible to use real market data rather than synthetic one, but from a development perspective, this is very drab. It would be slow to update, and I would be constrained to working in only market hours of the day/week. This is not at all to my benefit, but it may interest you to know it was considered.